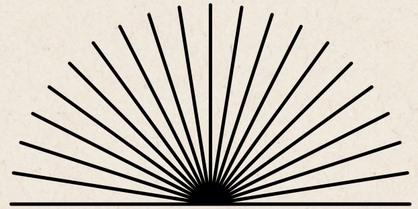
Universidade Federal de Ouro Preto - Campus Morro do Cruzeiro
Departamento de Computação - DECOM
Instituto de Ciências Exatas e Biológicas - ICEB

Uma apresentação sobre:
Byzantine Fault-Tolerant Consensus Algorithms.

FAST BFT

APRESENTADO POR:
Arlon Gabriel G. Machado.

DOCENTE:
Joubert de Castro Lima.



Por que o FASTBFT recebe esse nome?

É nomeado de FAST BFT pois foi justamente projetado para ser mais rápido que BFT’s tradicionais.

1. **Por que precisamos de BFT mais rápidos?**
2. **Ideia central do FASTBFT.**
3. **Arquitetura Geral.**
4. **Mecanismo de Secret Sharing e Agregação.**
5. **Fluxo do Protocolo (Fast Path).**
6. **Mudança de visão (View Change) e fallback.**
7. **Vantagens e Desempenho.**
8. **Limitações e desafios atuais.**

Por que precisamos de BFT mais rápidos?

- Protocolos clássicos (ex.: PBFT) sofrem com $O(n^2)$ mensagens.
- Alto custo de assinaturas digitais e verificações.
- Mudança de líder custosa e lenta.
- Baixo desempenho em sistemas com muitos nós.



Onde usar BFT's O(n)?

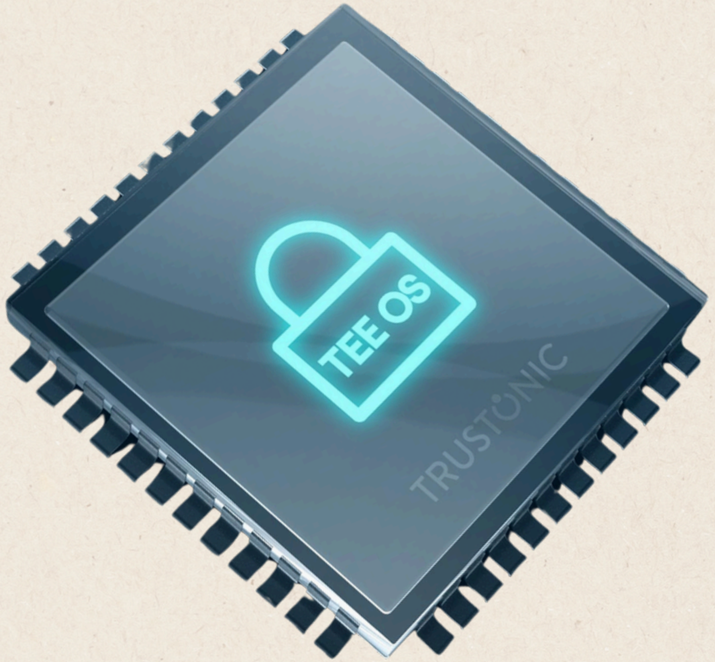
Blockchains Permissionadas.

Redes financeiras distribuídas entre bancos.

Ideia central do FASTBFT.

- Usa **Trusted Execution Environments (TEEs).**

1. Execução Segura e isolada do código crítico.
2. Armazenamento seguro de chaves e segredos.
3. Contador Monotônico.



CONTADOR MONOTÔNICO

O TEE mantém um contador interno.

Cada mensagem ou requisição que o nó envia é vinculada a um valor único e recém-incrementado deste contador monotônico

A monotonicidade do contador é assegurada pelo TEE.

Os nós não podem atribuir o mesmo valor de contador a mensagens diferentes

Por que TEE o torna mais rápido?

Sem o uso do TEE



Você precisa confiar no Sistema Operacional.



Réplicas podem forjar mensagens!



Para impedir, você usa assinaturas digitais pesadas - Para cada mensagem.



Aumento de custo, latência, complexidade e consumo de rede.

Usando o TEE



Você move a parte crítica para o Enclave TEE.



O TEE garante não equivocação - não vai enviar duas mensagens diferentes.



Gera shares leves ao invés de assinaturas pesadas.



O líder apenas agrega shares - sem verificar assinaturas.

Mecanismo de Secret Sharing e Agregação.

1	2	3	4
PRÉ PROCESSAMENTO (OFFLINE)	**RECEBE A REQUISIÇÃO LIGA AO CONTADOR**	**PREPARE**	**COMMIT/AGREGAÇÃO**
• TEE gera segredos aleatórios e publica o hash de cada segredo. • Divide cada segredo em shares e entrega cada share de forma segura ao TEE de cada réplica.	• Quando chega a requisição, o líder associa essa requisição a um segredo previamente preparado e ao valor monotônico do seu TEE. • Isso impede o uso do mesmo segredo para várias requisições.	• Cada réplica, ao receber o prepare com o contador correto, pede ao seu TEE para liberar sua share correspondente. • O TEE local só libera a share se o prepare trouxer o valor de contador certo	• Reconstrução do segredo: o líder (ou a árvore de agregação) coleta as shares, reconstrói o segredo e distribui o segredo reconstruído – os outros verificam usando o hash publicado no pré-processamento.

Fluxo do Protocolo (Fast Path)



- O líder é honesto!
- Não há falhas.
- Todas as réplicas respondem dentro do tempo esperado.
- **Reduz mensagens para O(n).**

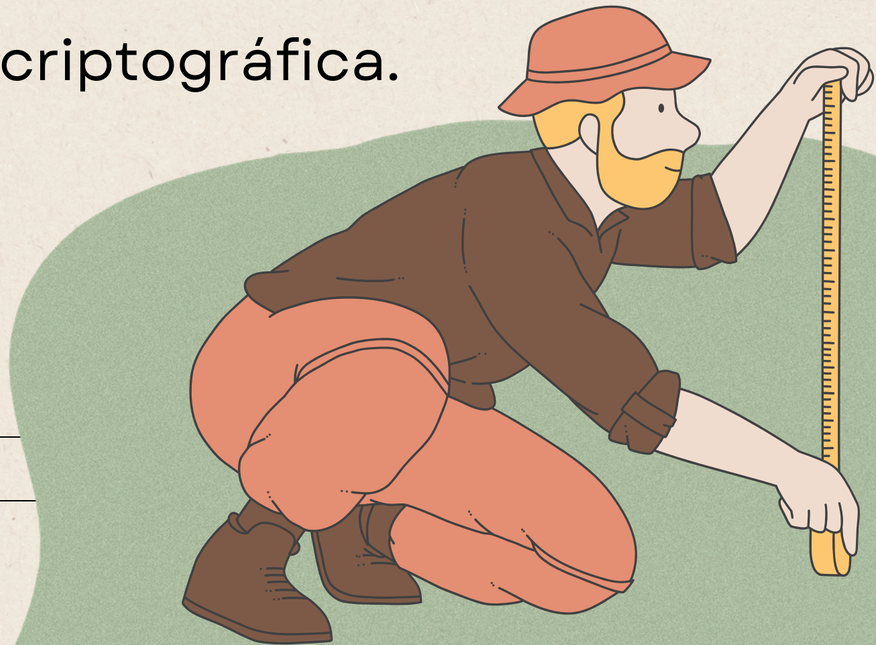
Mudança de visão (View Change) e fallback



1. **Falha detectada:** timeout, share inválida, hash incorreto ou falta de share.
2. **Envia SUSPECT:** réplicas avisam seus pais e o líder.
3. **Propagação da suspeita:** o líder identifica o nó/caminho problemático.
4. **Commit rejeitado:** qualquer commit/reply inválido é descartado.
5. **Inicia view-change:** réplicas coletam f+1 provas e elegem novo líder.
6. **Recovery (se réplicas reiniciam):** nó coleta **f+1** logs/valores para restaurar estado.
7. **TEE atualiza contador:** TEE só aceita contador consistente e valores verificados.
8. **Fallback ativado:** usa mais mensagens/verificações (modo mais lento e seguro).
9. **Reconstrução segura:** shares válidas são recolhidas e o segredo é reconstruído.
10. **Nova estabilidade:** novo líder envia certificado e todos voltam ao fast path.

Vantagens e Desempenho

- Comunicação O(n) → superior a PBFT ($O(n^2)$).
- Baixa latência com líder honesto.
- Maior numero de requisições que MinBFT, Zyzzyva e PBFT nos experimentos.
- TEE simplifica verificações e reduz complexidade criptográfica.
- Escala bem em clusters grandes.



Limitações e desafios atuais

Existe pseudo-códigos, mas não são baseados na lógica real do protocolo. E mesmo assim, os pseudocódigos não implementam o código interno do enclave, árvore de agregação e para os tipos de falhas.

- Forte dependência do Intel SGX (ou TEE equivalente).
- Vulnerável a ataques de canal lateral e limitações reais do SGX.
- Enclaves podem se tornar gargalos sob carga extrema.
- Exige infraestrutura com TEE disponível – não serve para blockchains públicas.
- Quase não existem implementações open-source completas.
- Existe somente em pesquisa, não em mercado.



Referências

1. **FastBFT** (arXiv v5 – 1612.04997v5) DU, Pengfei et al. FastBFT: High-performance Byzantine fault tolerance. arXiv preprintarXiv:1612.04997v5, 2017.

2. **FastBFT (versão revisada – Scalable Byzantine Consensus via Hardware-Assisted Secret Sharing)** GEORGIOU, Chryssis; SAVVA, Christoforos; TSIGAS, Philippas. Scalable Byzantine Consensus via Hardware-Assisted Secret Sharing. IEEE Transactions on Computers, 2020.

3. **Trusted Hardware-Assisted Leaderless BFT (2025-1033.pdf)** LI, Zhen; ZHANG, Yuxin; WANG, Wenbo; YANG, Fan. Trusted Hardware-Assisted Leaderless Byzantine Fault Tolerance. 2025.

4. **Achilles: Efficient TEE-Assisted Byzantine Consensus** (3689031.3717457.pdf) WU, X., et al. Achilles: Efficient TEE-Assisted Byzantine Fault Tolerant Consensus. Proceedings of the ACM SIGSAC Conference, 2022.

5. **electronics-12-03801.pdf (survey de BFT com hardware confiável)** GEORGIOU, Chryssis; SAVVA, Christoforos. Hardware-Assisted Byzantine Fault Tolerance: A Survey. Electronics, v. 12, n. 3801, 2023.

Universidade Federal de Ouro Preto - Campus Morro do Cruzeiro
Departamento de Computação - DECOM
Instituto de Ciências Exatas e Biológicas - ICEB

OBRIGADO!

Uma apresentação sobre:
Byzantine Fault-Tolerant Consensus Algorithms. (Fast BFT)

APRESENTADO POR:
Arlon Gabriel G. Machado.

DOCENTE:
Joubert de Castro Lima.